

Architecture Claude Code ? ? ? Vault & M ? ? ? moire

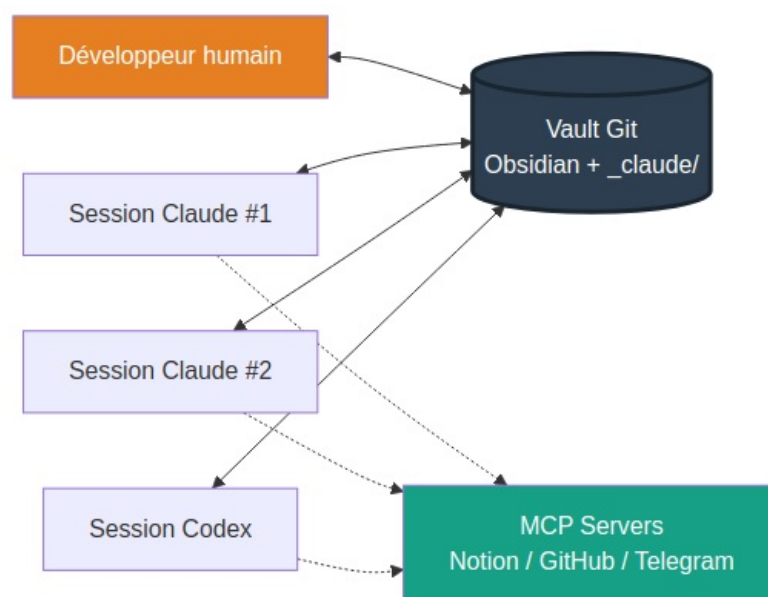
Vault partagé, mémoire persistante, workflow agentique

2026-04-23

1. Vue d'ensemble

Ce document décrit une architecture de travail entre un développeur humain et des agents IA (Claude Code, Codex, etc.) centrée sur un **vault partagé** qui sert de mémoire unique et persistante.

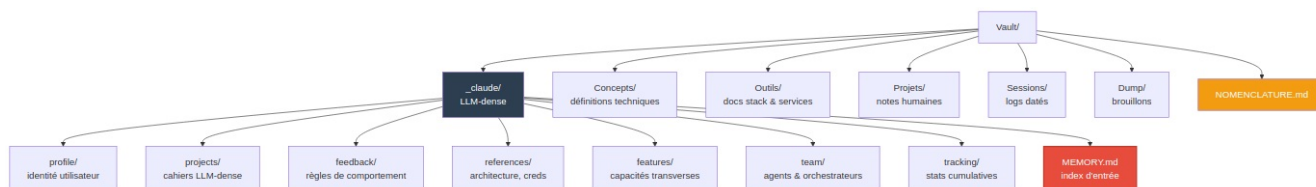
L'idée centrale : **un seul cerveau, plusieurs instances**. Chaque session d'IA commence en lisant la même source de vérité, et termine en la mettant à jour. Pas de dérive, pas de contexte perdu entre deux conversations.



2. Structure du vault

Le vault est un dépôt Git versionné, avec une **double couche** :

- **Couche humaine** (Concepts/ , Outils/ , Projets/ , Sessions/) → lisible en Obsidian, éditée manuellement.
- **Couche LLM-dense** (_claude/) → structurée pour l'IA, exclue du graphe Obsidian, lue via filesystem.



Règle fondamentale : les deux couches sont toujours synchronisées. Jamais l’une sans l’autre.

3. Nomenclature — préfixes

Chaque fichier porte un **préfixe** qui détermine son type, son emplacement et ses règles de cycle de vie.

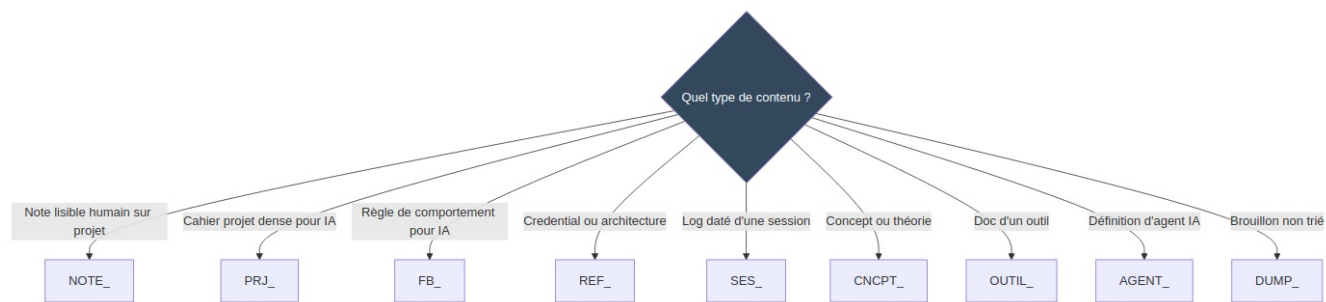
Règle universelle :

```
<PREFIX>_[<TEMPOREL>_]<slug>[_v<N>].<ext>
```

Préfixe	Type de contenu	Temporel	Dossier cible
PRJ_	Cahier projet LLM-dense	non	_claude/projects/
NOTE_	Note projet humaine	non	Projets/
FB_	Règle de comportement IA	non	_claude/feedback/
REF_	Credential / architecture / token	non	_claude/references/
USR_	Profil utilisateur	non	_claude/profile/
SES_	Log de session daté	oui	Sessions/
COMPACT_	Sauvegarde pré-compaction contexte	oui	Sessions/_compact/
CNCPT_	Concept / théorie / apprentissage	non	Concepts/
OUTIL_	Documentation d’un outil / stack	non	Outils/
FEAT_	Fonctionnalité transversale	non	_claude/features/
AGENT_	Définition d’un agent IA	non	_claude/team/
TEAM_	Orchestrateur d’agents	non	_claude/team/
CDC_	Cahier des charges projet	non	dossier projet
DUMP_	Brouillon non classifié	oui	Dump/

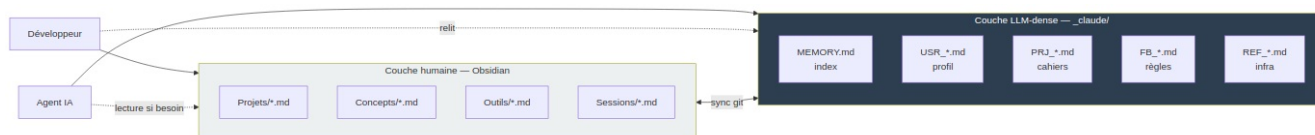
Slug : minuscules, underscores, ASCII pur. Pas d’accents, pas d’espaces, pas de tirets.

Arbre de décision simplifié :



4. Double couche de mémoire

La séparation entre couche humaine et couche LLM-dense est ce qui permet à plusieurs agents IA de se partager un contexte sans polluer la vue de l'humain.

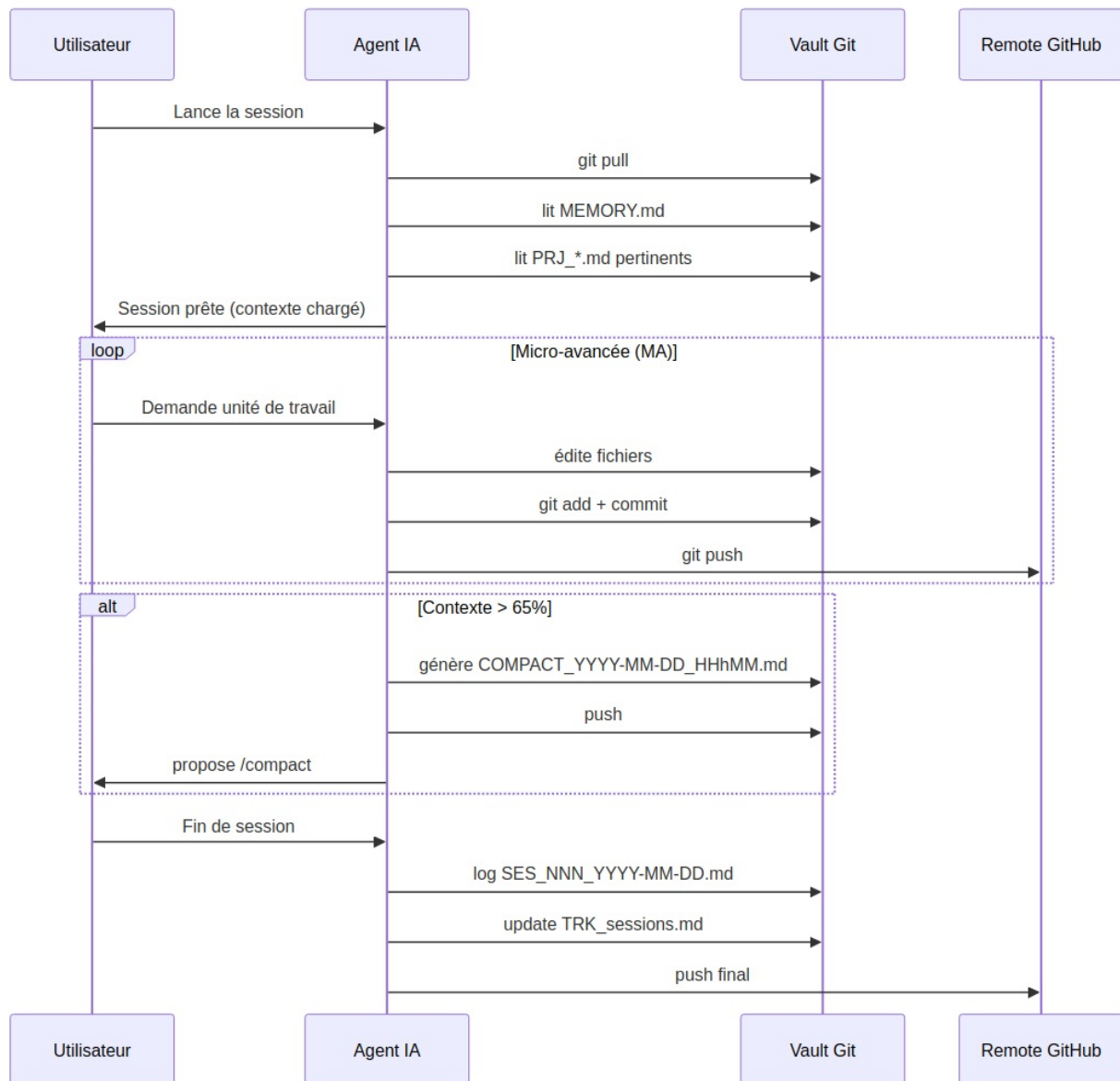


Caractéristiques de la couche LLM-dense :

- Pas de fioritures narratives — uniquement des faits structurés et actionnables.
- Chaque fichier a un frontmatter YAML typé (`type:` , `slug:` , `updated:` , `projet:`).
- Les liens entre fichiers sont explicites (paths relatifs) pour être parsables par un graph.
- Lue directement via filesystem, pas via le moteur Obsidian.

5. Cycle de vie d'une session IA

Une session suit un protocole strict : démarrage, travail en micro-avancées committées, compaction du contexte si besoin, fin de session.

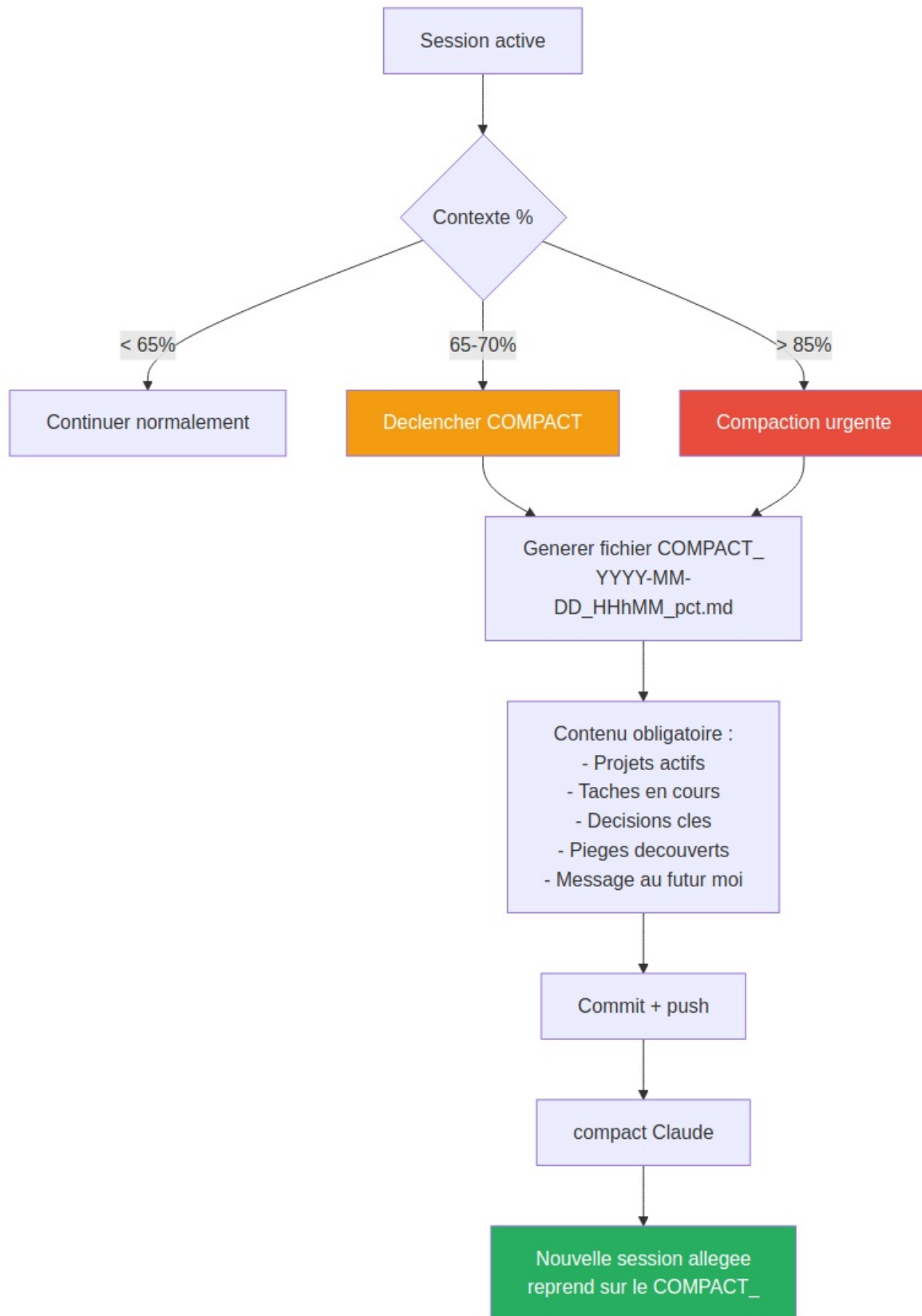


Principes du workflow :

- **Pull avant tout** → jamais de divergence entre instances.
- **Commit par micro-avancée** → jamais de gros commits batch, chaque unité de travail atomique est traçable.
- **Push systématique** → l'état est toujours récupérable depuis n'importe quelle machine.

6. Protocole de compaction

Le contexte d'un modèle LLM est une ressource limitée. Pour ne pas attendre la saturation, un protocole de compaction est déclenché proactivement.



Contenu minimal d'un COMPACT :

```
---
type: compact
context_pct: 68
date: 2026-04-23
session_topics: [sujet1, sujet2]
---
```

```

## Projets actifs
- <projet> : <état en 1 ligne>

## Tâches en cours
- [ ] <tâche>

## Décisions clés de la session
- <décision>

## Pièges découverts
- <piège>

## À dire à la prochaine instance
<ce qu'il faut savoir pour reprendre sans friction>

```

7. Environnement d'exécution — containers Docker

Chaque contexte de travail tourne dans un **container Docker isolé** : réseau séparé, identité git propre, aucun volume monté (tout vit à l'intérieur du container).

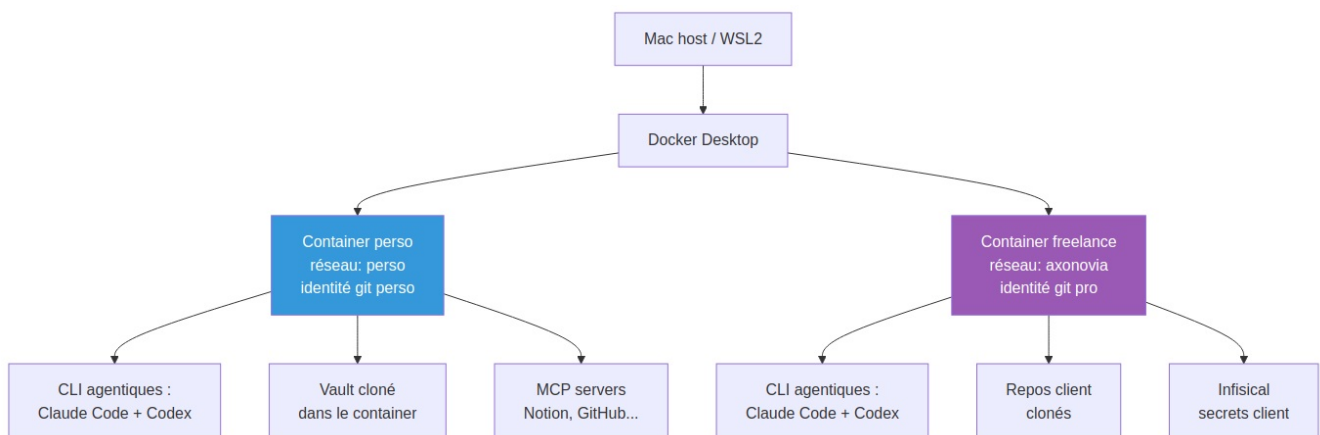


Image de base (Node 20 slim) :

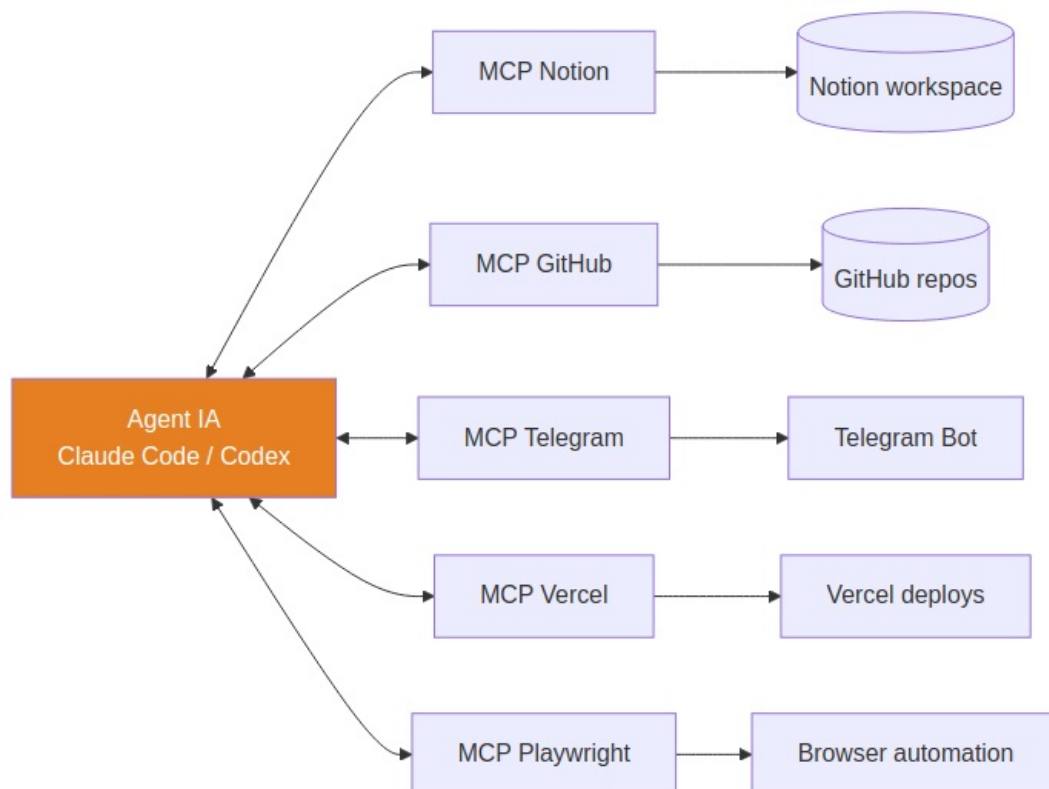
- `git`, `openssh-client`, `curl`, `ripgrep`, `vim`, `less`, `sudo`
- CLI agentiques installés globalement (`@anthropic-ai/claude-code`, `@openai/codex`)
- User non-root `dev` avec `sudo` sans mot de passe

Règles d'isolation :

- **Pas de** `docker compose down` → effacerait le contenu du container.
- `stop` / `start` sont sûrs → tout l'état persiste.
- **Secrets via Infisical** dans le container, jamais en `.env` en clair.

8. Intégrations MCP

Les agents IA communiquent avec le monde extérieur via **MCP servers** (Model Context Protocol) — des bridges vers des services.



Les MCP sont déclarés dans `~/.claude.json` (Claude Code) ou `~/.codex/config.toml` (Codex). Ajouter un MCP = éditer le fichier, pas rebuild l'image.

9. Workflow Git



Règles :

- Branche `main` = source de vérité stable.
- Branches `claude/<slug>` = travail d'une session ou d'une feature.
- **Un commit par micro-avancée**, jamais de batch.
- Messages en format impératif : `docker: rename X to Y`, `session: start SID=abc`, `feedback: add rigor rule`.

10. Règles de comportement pour l'IA

Des fichiers `FB_*.md` encodent les règles de comportement attendues. Voici les plus structurantes :

Règle	Principe
Rigueur absolue	Jamais de validation approximative. <i>"Je ne sais pas"</i> > invention.
Pas de yes-man	Challenger avant d'exécuter. Corriger explicitement une erreur utilisateur.
Pas de confirmation	Agir directement quand le contexte est clair.
Micro-avancées	Commiter après chaque unité atomique de travail.
Documenter pour le futur	Jamais <i>"c'est évident"</i> — penser à la prochaine instance.
Quality gate 8/10	Ne pas dégrader la précision pour économiser des tokens.
Check docs first	Lire la documentation existante avant de poser une question.

Ces règles sont chargées automatiquement à chaque session via `MEMORY.md`.

11. Principes fondateurs

1. **Source de vérité unique** → le vault Git. Pas d'info critique ailleurs.
2. **Réversibilité** → tout est versionné, tout est récupérable.
3. **Séparation des couches** → humain ↔ LLM-dense, jamais mélangés.
4. **Micro-avancées commitées** → aucun travail ne doit être perdu sur un crash.
5. **Isolation par contexte** → un container par client/projet, identités séparées.
6. **Moindre privilège** → chaque outil a l'accès minimum nécessaire (ex : bot Telegram vs compte MTPROTO complet).
7. **Rigueur > vitesse** → mieux vaut une session lente et exacte qu'une session rapide et approximative.

Document généré automatiquement depuis le vault — version neutralisée pour partage externe.